



MediFlow - A Healthcare Patient Management System

COMP41720 Distributed Systems - Group Project Report

Module Co-ordinator: Dr. Avishek Nag

Academic Year: 2025-26 Autumn Trimester

Repository Link: <https://github.com/vijethph/MediFlow>

Team Members:

- Vijeth Prashanth Hegde - 25202022 - vijeth.prashanthhegde@ucdconnect.ie
- Akshay Channapla Udaya Kumar - 25211966 - akshay.channaplaudayakumar@ucdconnect.ie
- Sai Deep Kandra - 25207532 - sai.kandra@ucdconnect.ie
- Lakshmi Kiran Chikka Hosagavi Manchaiah - 25207663 - lakshmi.chikkahosagavimanc@ucdconnect.ie

Table of Contents

MediFlow - A Healthcare Patient Management System	1
1. Introduction	3
1.1 Project Context and Objectives	3
1.2 Application Domain: Healthcare Management	3
1.3 Technology Stack and Rationale	3
1.4 FHIR Compatibility Approach	4
1.5 Architectural Overview	4
2. System Design and Architecture	5
2.1 Service Decomposition Strategy	5
2.2 Communication Patterns and Data Flow	9
2.3 Data Management and Persistence	11
2.4 Infrastructure Components	13
3. Architectural Decision Records (ADRs)	15
3.1 ADR-001: Database Per Service Pattern	15
3.2 ADR-002: Hybrid Communication Patterns (Synchronous REST + Asynchronous Messaging)	16
3.3 ADR-003: JWT Authentication with Centralized Gateway Validation	18
4. Monitoring and Observability	20
4.1 Grafana Dashboards	20
4.2 Observability Best Practices Implemented	21
5. Student Contributions	22
5.1 Sai Deep Kandra (25207532): Patient Management Service	22
5.2 Lakshmi Kiran Chikka Hosagavi Manchaiah (25207663): Appointment Scheduling Service	22
5.3 Akshay Channapla Udaya Kumar (25211966): Prescription Management Service	23
5.4 Vijeth Prashanth Hegde (25202022): Billing Service and Monitoring Infrastructure	23
5.5 Collaborative Work	23
6. Trade-off Analysis and Lessons Learned	23
6.1 Major Trade-offs Revisited	24
6.2 Performance Considerations	25
6.3 Key Takeaways	25
7. Conclusion	26
7.1 Objectives Achieved	26
7.2 Key Technical Contributions	26
7.3 Learning Outcomes	27
7.4 Project Scope and Limitations	27
7.5 Final Reflection	27

1. Introduction

1.1 Project Context and Objectives

This project implements **MediFlow - a Healthcare Patient Management System** as a distributed microservices application, demonstrating practical application of architectural principles and trade-off analysis in distributed systems design.

Core Objectives:

- Design and implement a distributed system with appropriate service boundaries
- Demonstrate hybrid communication patterns (synchronous and asynchronous)
- Deploy using cloud-native technologies (Docker, Kubernetes)
- Implement fault tolerance mechanisms (retries, circuit breakers, health checks)
- Apply security principles (JWT authentication, API Gateway pattern)
- Document architectural decisions with explicit trade-off analysis
- Create a reproducible system that can be deployed in multiple environments

The project scope intentionally balances **complexity with feasibility**, focusing on demonstrating core distributed systems concepts rather than production-grade completeness.

1.2 Application Domain: Healthcare Management

The Healthcare Patient Management System addresses the complex domain of healthcare operations, managing four critical business functions:

1. Patient Management
2. Appointment Scheduling
3. Prescription Management
4. Billing and Claims

1.3 Technology Stack and Rationale

The technology choices were made with consideration for **rapid development**, **strong ecosystem support**, and **alignment with distributed systems principles**:

Core Technologies

- **Python 3.11** - Selected for its extensive library ecosystem, strong async support
- **FastAPI** - Chosen for built-in Pydantic validation, native async/await support
- **RabbitMQ** - AMQP-based message broker selected for reliability, routing flexibility
- **Prometheus** - Industry-standard metrics collection with PromQL query language
- **Grafana** - Visualization and dashboarding with rich Prometheus integration
- **Tenacity** - Retry library with exponential backoff, jitter, and flexible retry conditions
- **Next.js 14** - React-based framework with App Router, server components
- **TypeScript** - Type-safe development for frontend codebase
- **Tailwind CSS v4** - Utility-first CSS framework for responsive UI design
- **React Query** - Asynchronous state management with caching and automatic refetching
- **Zod** - Schema validation for forms and API responses

Infrastructure Technologies

- **Docker** - Multi-stage builds with `python:3.11-slim` base images for security and size optimization

- **Docker Compose** - Multi-container orchestration for local full-stack testing
- **Kubernetes** - Production-grade container orchestration tested with Minikube locally
- **Kong**- Open-source API Gateway in DB-less mode (declarative configuration), providing centralized JWT validation, request routing, and rate limiting capabilities
- **PostgreSQL 17** - ACID-compliant relational database for Patient, Appointment, and Billing services requiring transactional integrity
- **MongoDB 6** - Document database for Prescription Service, accommodating flexible clinical document schemas
- **Redis** - In-memory cache for reducing database load and improving response times

1.4 FHIR Compatibility Approach

FHIR (Fast Healthcare Interoperability Resources) is an HL7 standard for electronic health information exchange. While full FHIR compliance was not a requirement, schemas were designed to be **FHIR R4-compatible** to demonstrate:

1. **Industry Alignment:** Understanding real-world healthcare interoperability standards
2. **Extensibility:** Facilitating future integration with FHIR-compliant systems
3. **Structured Data Modeling:** Learning standardized approaches to complex domain modeling

FHIR Resources Implemented:

FHIR Resource	Service	Compatibility Level
Patient	Patient Service	High - Includes HumanName, ContactPoint, Address, identifier
Appointment	Appointment Service	High - Includes status, participant, serviceType, start/end times
MedicationRequest	Prescription Service	Medium - Core fields implemented, dosageInstruction supported
Invoice	Billing Service	High - Includes status, lineItem, totalNet/totalGross, subject reference

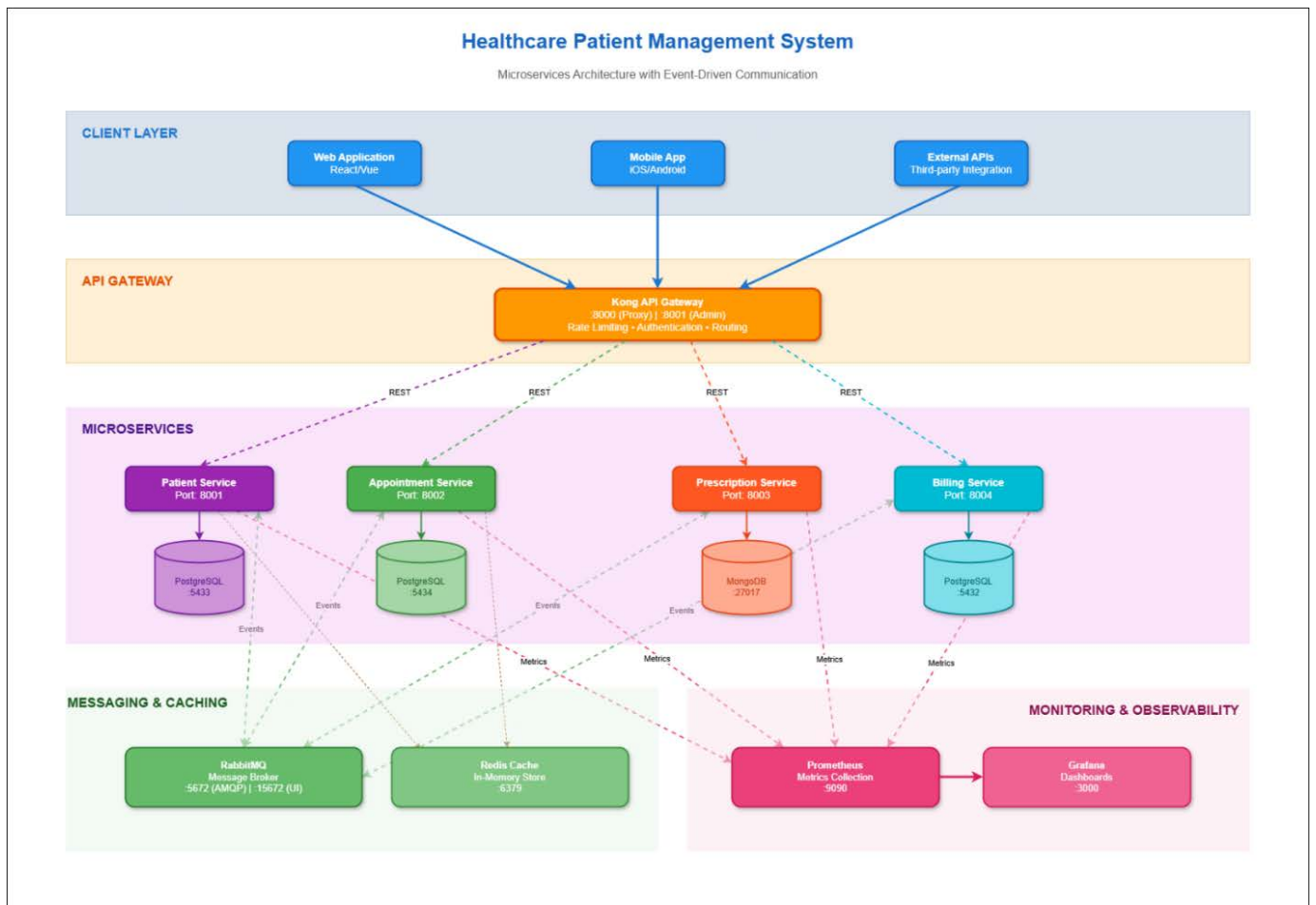
Key FHIR Patterns Applied:

- **Resource Type Field:** Every entity includes resource_type (e.g., "Patient", "Invoice")
- **Status Enums:** FHIR-compliant status values (e.g., Invoice: draft, issued, balanced, cancelled)
- **References:** Cross-service references use URIs or IDs (e.g., subject: patient_id)
- **Metadata:** meta JSONB column for FHIR metadata (version, lastUpdated, profile)

1.5 Architectural Overview

The system implements a **microservices architecture** with the **Database-per-Service** pattern, where each service owns its domain and data. This design choice prioritizes **service autonomy** and **independent scalability** over data consistency simplicity.

System Topology:



Architecture Layers:

- API Gateway Layer:** Kong provides single entry point, centralizing cross-cutting concerns (authentication, logging, rate limiting)
- Microservices Layer:** Four autonomous services with clear bounded contexts
- Infrastructure Layer:** Shared messaging (RabbitMQ) and caching (Redis) services
- Data Layer:** Each service has dedicated database (PostgreSQL or MongoDB)
- Monitoring Layer:** Prometheus for metrics collection, Grafana for visualization

Communication Patterns:

- Solid Lines:** Service ownership and data access
- Dashed Lines:** Inter-service communication (REST APIs)
- Message Flow:** Asynchronous event-driven communication via RabbitMQ

This architecture will be explored in detail in Section 2, with explicit trade-off analysis documented in Section 3 (ADRs).

2. System Design and Architecture

2.1 Service Decomposition Strategy

Service boundaries were defined following **Domain-Driven Design (DDD)** principles, where each service represents a **bounded context** with cohesive business capabilities. The decomposition

balances **granularity disintegrators** (factors favoring smaller services) and **granularity integrators** (factors favoring larger services).

2.1.1 Service Boundaries and Responsibilities

Patient Management Service (Port 8001)

Bounded Context: Patient identity, demographics, and medical history

Core Responsibilities:

- Patient registration and profile management
- Authentication (JWT token generation)
- Medical history CRUD operations
- Patient search and retrieval
- Consent and privacy preferences

Database: PostgreSQL (ACID properties for identity data)

API Endpoints:

- `POST /api/v1/patients/register` - Create patient account
- `POST /api/v1/patients/login` - Authenticate and issue JWT
- `GET /api/v1/patients/{id}` - Retrieve patient details
- `PUT /api/v1/patients/{id}` - Update patient profile
- `GET /api/v1/patients/search` - Search patients by criteria
- `POST /api/v1/patients/{id}/history` - Add medical history entry

Events Published:

- `patient.registered` - New patient account created
- `patient.updated` - Patient profile modified
- `patient.deleted` - Patient account deactivated

Appointment Scheduling Service (Port 8002)

Bounded Context: Doctor calendars, appointment slots, and scheduling workflow

Core Responsibilities:

- Appointment slot creation and availability management
- Appointment booking with conflict detection
- Appointment cancellation and rescheduling
- Schedule queries by patient, doctor, or date range
- Waitlist management (future extension)

Database: PostgreSQL (transactional integrity for booking operations)

API Endpoints:

- `POST /api/v1/appointments` - Book appointment
- `GET /api/v1/appointments/{id}` - Get appointment details
- `PUT /api/v1/appointments/{id}` - Update appointment
- `DELETE /api/v1/appointments/{id}` - Cancel appointment
- `GET /api/v1/appointments/patient/{patient_id}` - List patient appointments

- GET /api/v1/appointments/availability - Check slot availability

Events Published:

- appointment.created - New appointment booked (triggers invoice generation)
- appointment.cancelled - Appointment cancelled (triggers refund workflow)
- appointment.rescheduled - Appointment time changed

Events Consumed:

- patient.deleted - Cascade cancel all patient appointments

Inter-Service Dependencies:

- **Synchronous:** Calls Patient Service to verify patient exists before booking (REST API)
- **Asynchronous:** Receives patient deletion events via RabbitMQ

Prescription Management Service (Port 8003)

Bounded Context: Clinical documentation, medication orders, and prescriptions

Core Responsibilities:

- Prescription creation and management
- Medication history tracking
- Drug interaction warnings (future: integrate with external drug database)
- Prescription refill requests
- Lab result storage (flexible schema)

Database: MongoDB (flexible schema for diverse clinical documents)

API Endpoints:

- POST /api/v1/prescriptions - Create prescription
- GET /api/v1/prescriptions/{id} - Get prescription details
- GET /api/v1/prescriptions/patient/{patient_id} - List patient prescriptions
- PUT /api/v1/prescriptions/{id} - Update prescription
- POST /api/v1/prescriptions/{id}/refill - Request refill

Events Published:

- prescription.created - New prescription issued
- prescription.filled - Prescription dispensed
- prescription.refill_requested - Refill authorization needed

Inter-Service Dependencies:

- **Synchronous:** Calls Patient Service to verify patient exists before creating prescription

Billing and Claims Service (Port 8004)

Bounded Context: Financial transactions, invoicing, and insurance claims

Core Responsibilities:

- Invoice generation from appointments
- Payment processing and tracking
- Insurance claim submission (simulated)

- Refund processing
- Financial reporting and analytics

Database: PostgreSQL (financial data requires ACID guarantees)

API Endpoints:

- POST /api/v1/invoices - Create invoice (usually automated via event)
- GET /api/v1/invoices/{id} - Get invoice details
- GET /api/v1/invoices/patient/{patient_id} - List patient invoices
- POST /api/v1/payments - Record payment
- GET /api/v1/payments/{id} - Get payment details
- POST /api/v1/claims - Submit insurance claim
- GET /api/v1/claims/{id} - Get claim status

Events Published:

- `invoice.created` - Invoice generated
- `payment.received` - Payment recorded
- `claim.submitted` - Insurance claim sent
- `refund.processed` - Refund completed

Events Consumed:

- `appointment.created` - Automatically generate invoice
- `appointment.cancelled` - Initiate refund workflow

Inter-Service Dependencies:

- **Synchronous:** Calls Appointment Service to get appointment details for invoice creation
- **Asynchronous:** Listens to appointment events via RabbitMQ

2.1.2 Granularity Analysis

Disintegrators (Favoring Smaller Services):

Factor	Impact	Services Affected
Code Volatility	Billing rules change frequently due to insurance policy updates	Billing Service isolated
Scalability Differences	Appointment queries spike during business hours, billing batch processing at night	Independent scaling
Technology Optimization	MongoDB for clinical documents, PostgreSQL for transactional data	Prescription vs. others
Fault Isolation	Patient service failure shouldn't prevent appointment queries	All services

Integrators (Favoring Larger Services):

Factor	Impact	Mitigation Strategy
Data Joins	Cannot SQL join across patient, appointment, billing data	API composition pattern, denormalization where needed
Distributed	Cannot use database transactions across	Saga pattern with compensating

Transactions	services	transactions
Network Latency	Inter-service calls add overhead	Caching (Redis), batch APIs, async events
Operational Complexity	4 databases to manage, backup, monitor	Docker Compose and Kubernetes automation
Testing Complexity	Integration tests require multiple services running	Test containers, service mocking

Conclusion: The granularity represents a **pragmatic middle ground**, avoiding both the monolith extreme (single service) and nano-services extreme (separate services for every entity). Each service aligns with a natural business capability and team member.

2.2 Communication Patterns and Data Flow

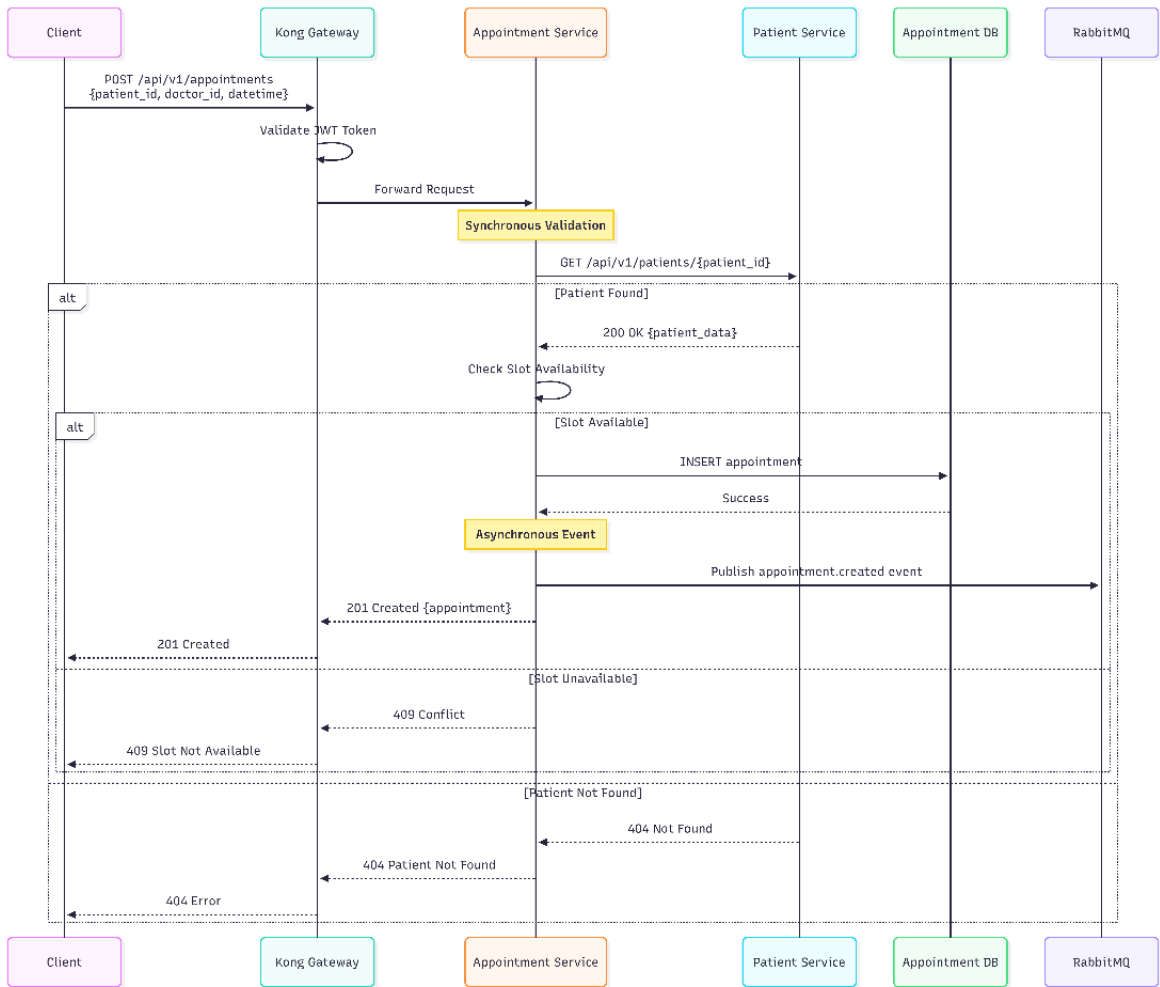
The system employs a **hybrid communication architecture** combining synchronous REST APIs and asynchronous message-based communication.

2.2.1 Synchronous REST Communication

Use Cases:

- User-facing queries requiring immediate response
- Data validation before operations (e.g., verify patient exists before booking appointment)
- Real-time data retrieval

Example Flow: Appointment Creation with Patient Validation



Trade-offs:

- **✓ Immediate Consistency:** Patient validation ensures appointment created only for valid patients
- **✓ User Feedback:** Client receives immediate response on success/failure
- **✗ Tight Coupling:** Appointment Service depends on Patient Service availability
- **✗ Latency:** Network call to Patient Service adds 50-200ms overhead

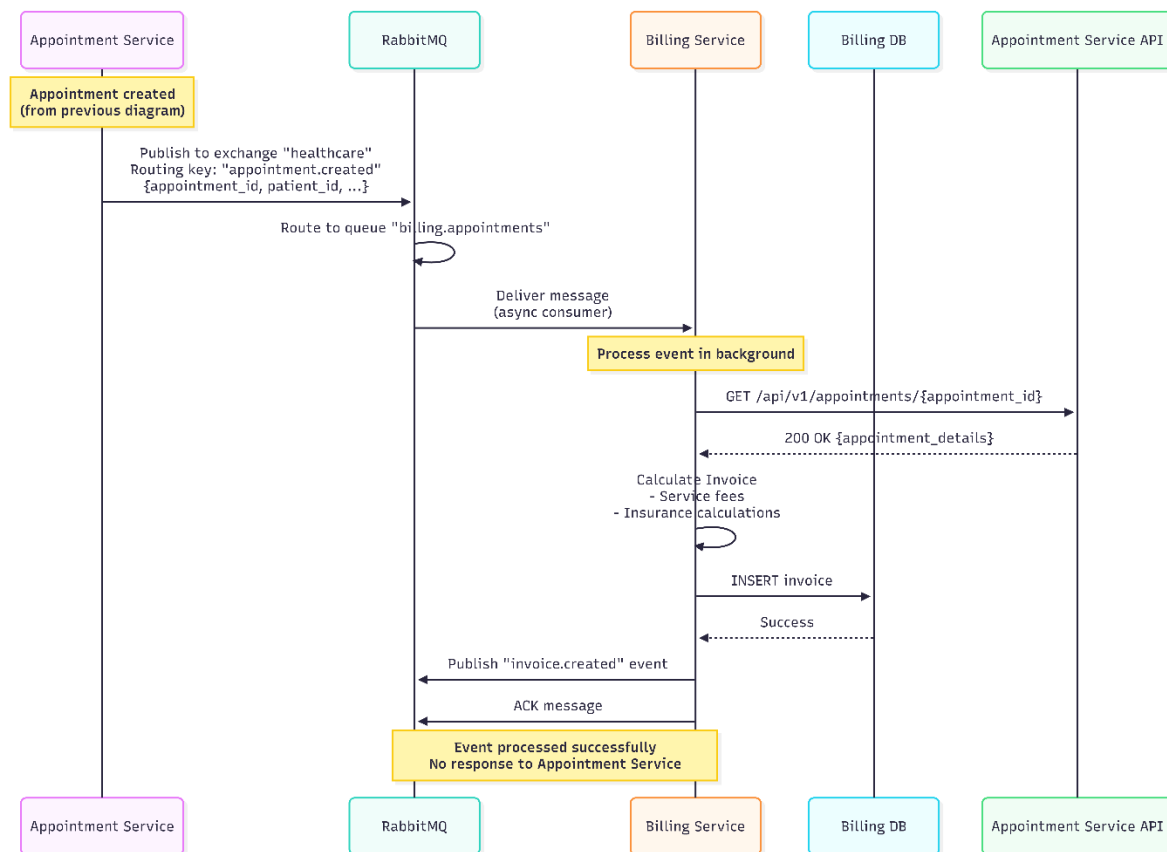
Mitigation: Implement retry logic with exponential backoff using Tenacity library.

2.2.2 Asynchronous Message-Based Communication

Use Cases:

- Event notifications between services
- Background processing (invoice generation, reports)
- Decoupled workflows where immediate response not required
- Long-running operations

Example Flow: Invoice Generation After Appointment



Trade-offs:

- **✓ Loose Coupling:** Services don't need to know about each other
- **✓ Resilience:** Messages persist if Billing Service is down (durable queues)
- **✓ Scalability:** Multiple Billing Service instances can consume from same queue
- **✗ Asynchronous Processing:** Appointment creation doesn't wait for invoice generation
- **✗ Eventual Consistency:** Invoice created seconds/minutes after appointment
- **✗ Debugging Complexity:** Harder to trace event flow across services
- **✗ Duplicate Processing Risk:** Must implement idempotency (same event processed twice produces same result)

Mitigation:

- Idempotency: Invoice includes appointment_id as unique constraint (duplicate events rejected by database)
- Dead Letter Queue: Failed messages moved to DLQ for manual inspection
- Monitoring: Prometheus metrics track message processing lag

2.3 Data Management and Persistence

2.3.1 Database-per-Service Pattern

Each microservice has its own dedicated database instance, enforcing strict **data ownership** and **service autonomy**. This pattern represents a fundamental trade-off: **service independence vs. data consistency simplicity**.

Database Allocation:

Service	Database	Technology	Port	Rationale
Patient	patient_db	PostgreSQL 17	5432	ACID for identity data, structured schema
Appointment	appointment_db	PostgreSQL 17	5433	Transactional integrity for booking
Prescription	prescription_db	MongoDB 6	27017	Flexible schema for clinical documents
Billing	billing_db	PostgreSQL 17	5435	Financial data requires ACID guarantees

Technology Choice Justification:

PostgreSQL for Patient/Appointment/Billing:

- ACID transactions ensure data integrity (critical for bookings, payments)
- Strong consistency guarantees
- SQL queries for complex joins and reports
- Foreign key constraints for referential integrity

MongoDB for Prescription:

- Flexible schema accommodates diverse clinical document types (prescriptions, lab results, imaging reports)
- JSON document structure aligns with FHIR resources
- Easy to extend with new fields without schema migrations
- Full-text search capabilities for clinical notes
- Horizontal scaling for large clinical data volumes

This represents a **polyglot persistence** approach, choosing the right tool for each domain’s specific needs.

2.3.2 Database Schema Examples

Patient Service (PostgreSQL):

Column Name	Data Type	Constraints	Description
id	UUID	PRIMARY KEY, DEFAULT	Auto-generated unique identifier
resource_type	VARCHAR(50)	DEFAULT ‘Patient’	FHIR resource type

identifier	VARCHAR(100)	UNIQUE, NOT NULL	MRN or SSN
active	BOOLEAN	DEFAULT true	Account active status
given_name	VARCHAR(100)	NOT NULL	Patient's first/given name
family_name	VARCHAR(100)	NOT NULL	Patient's last/family name

Prescription Service (MongoDB):

The Prescription Service uses MongoDB to store FHIR MedicationRequest documents with flexible schemas:

Field	Type	Description
_id	ObjectId	MongoDB unique identifier
resourceType	String	"MedicationRequest"
status	String	active, completed, cancelled
subject	String	Patient ID reference
encounter	String	Appointment ID reference
medication	Object	CodeableConcept with coding system, code, display
dosageInstruction	Array	Dosage timing, route, dose quantity
requester	String	Doctor ID (prescriber)
dispenseRequest	Object	Number of refills, quantity
authoredOn	ISODate	Prescription creation timestamp
meta	Object	Version ID, last updated timestamp

The flexible document structure accommodates varying clinical data types including lab results, imaging reports, and prescription details without schema migrations.

2.3.3 Data Consistency Challenges and Solutions

The Database-per-Service pattern introduces **distributed data management challenges**:

Challenge 1: Cross-Service Queries

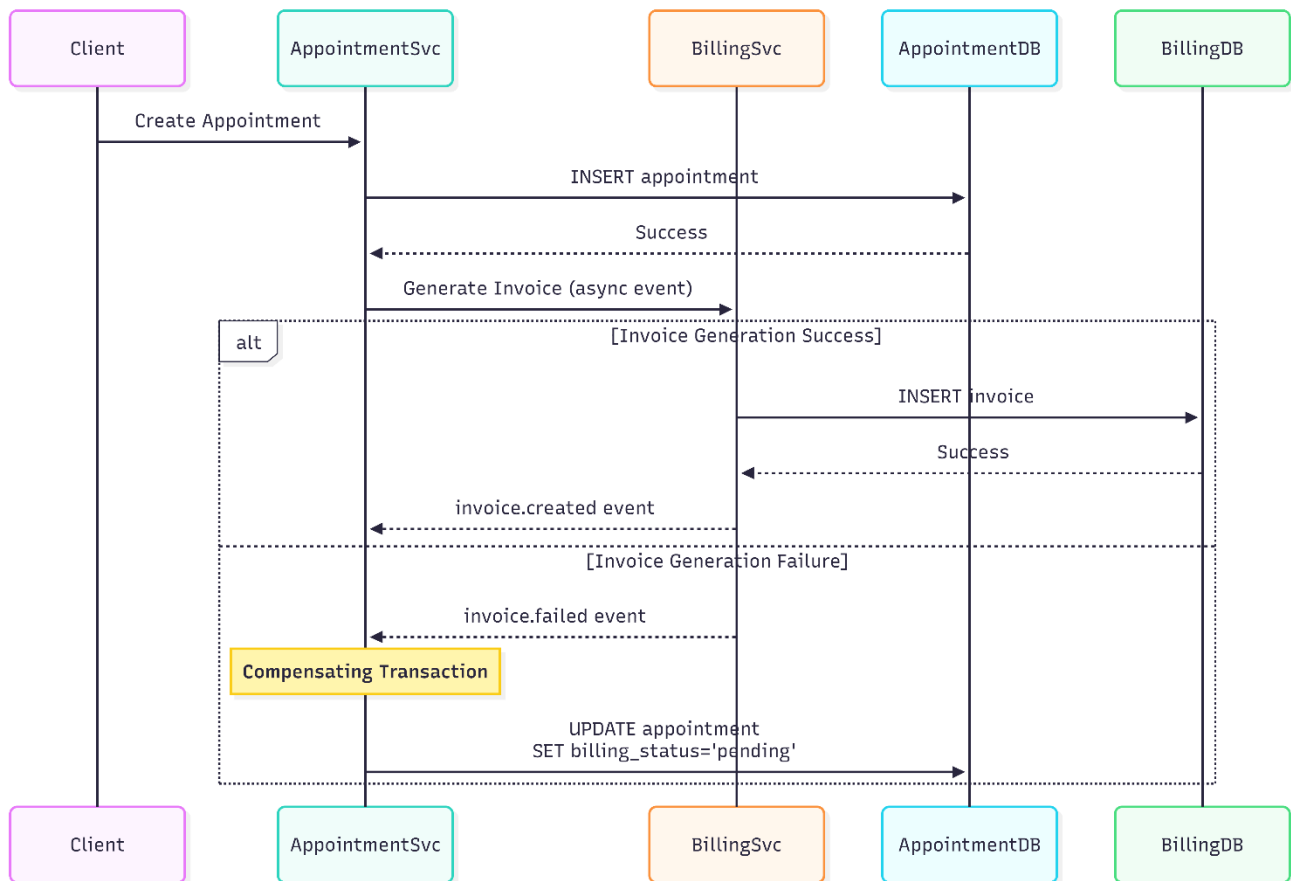
Problem: Cannot SQL join across patient, appointment, and billing data.

Solution: **API Composition Pattern**

Challenge 2: Distributed Transactions

Problem: Cannot use database ACID transactions across services (e.g., create appointment + generate invoice atomically).

Solution: **Saga Pattern with Compensating Transactions**



Challenge 3: Data Duplication

Problem: Services may need to cache data from other services (e.g., Billing caches patient names for invoices).

Solution: **Event-Driven Data Synchronization + Redis Caching**

2.4 Infrastructure Components

2.4.1 Kong API Gateway

Kong serves as the **single entry point** for all external client requests, implementing the **API Gateway pattern**.

Responsibilities:

- **Request Routing:** Routes requests to appropriate backend services based on URL path
- **Authentication:** Validates JWT tokens centrally (services trust pre-validated tokens)
- **Rate Limiting:** Prevents abuse and DDoS attacks
- **CORS Handling:** Manages cross-origin requests for web clients
- **Request/Response Transformation:** Adds correlation IDs, formats responses
- **Metrics:** Exposes Prometheus metrics for gateway performance

Trade-offs:

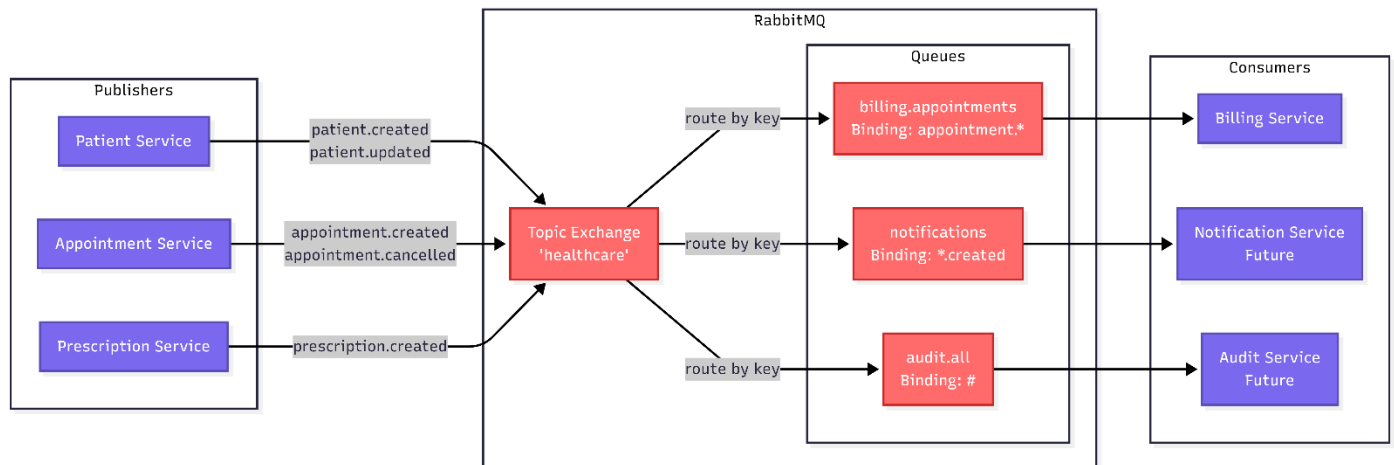
- **✓ Single Entry Point:** Clients only need to know gateway URL
- **✓ Centralized Security:** JWT validation in one place
- **✓ Service Abstraction:** Backend services can change without client updates
- **✓ Cross-Cutting Concerns:** Logging, rate limiting handled centrally

- **✗ Single Point of Failure:** Gateway downtime affects all services (mitigated with HA deployment)
- **✗ Latency:** Additional network hop adds 10-50ms
- **✗ Complexity:** Another component to configure and monitor

2.4.2 RabbitMQ Message Broker

RabbitMQ implements the **Event-Driven Architecture** pattern, enabling asynchronous communication between services.

Architecture:



Message Durability:

- **Durable Exchanges:** Survive broker restarts
- **Durable Queues:** Messages persist to disk
- **Persistent Messages:** Individual messages marked as persistent
- **Acknowledgments:** Consumers must acknowledge message processing before removal

Trade-offs:

- **✓ Decoupling:** Publishers don't know about consumers
- **✓ Reliability:** Messages not lost if consumer is down
- **✓ Scalability:** Multiple consumers can process messages in parallel
- **✓ Flexibility:** Add new consumers without modifying publishers
- **✗ Eventual Consistency:** Events processed with delay (seconds to minutes)
- **✗ Complexity:** Requires understanding of exchanges, queues, bindings
- **✗ Ordering Challenges:** Message order not guaranteed across multiple consumers

2.4.3 Redis Cache

Redis provides **in-memory caching** to reduce database load and improve response times.

Use Cases:

1. **Session Storage:** JWT token blacklist (for logout functionality)
2. **Query Result Caching:** Frequently accessed patient/appointment data
3. **Rate Limiting:** Track API request counts per user
4. **Distributed Locking:** Prevent double-booking of appointment slots

Trade-offs:

- **✓ Performance:** Sub-millisecond read latency vs. 10-100ms database queries

- ☒ **Reduced Database Load:** Offloads read-heavy workloads
 - ☒ **Scalability:** Handles high request volumes
 - ☒ **Cache Invalidation Complexity:** Must invalidate on updates (hardest problem in CS)
 - ☒ **Memory Constraints:** Limited by RAM (eviction policies needed)
 - ☒ **Stale Data Risk:** Cache may serve outdated data if invalidation missed
-

3. Architectural Decision Records (ADRs)

This section documents the three most significant architectural decisions:

3.1 ADR-001: Database Per Service Pattern

Status: Accepted

Date: November 2025

Context

In our Healthcare Patient Management System with 4 microservices (Patient, Appointment, Prescription, Billing), we needed to decide how to manage data. The fundamental question: **shared database or database-per-service?**

Key Considerations:

- **Team Structure:** 4 students working independently on separate services
- **Independent Deployment:** Services should deploy without coordinating database migrations
- **Technology Flexibility:** Different data models (relational for transactions, document for clinical records)
- **Fault Isolation:** Database failures should not cascade across services
- **Data Consistency:** Some operations require cross-service data (e.g., verify patient exists before appointment)
- **Operational Complexity:** Managing multiple databases increases ops burden

Alternatives Considered:

1. **Shared Database:** Single database with tables for all services
2. **Database per Service:** Each service owns its database
3. **Hybrid:** Core entities in shared database, service-specific data separated

Decision

We chose **Database Per Service** pattern with the following allocation:

- **Patient Service:** PostgreSQL (patient_db) - ACID for identity data
- **Appointment Service:** PostgreSQL (appointment_db) - Transactional integrity for scheduling
- **Prescription Service:** MongoDB (prescription_db) - Flexible schema for clinical documents
- **Billing Service:** PostgreSQL (billing_db) - Financial transactions require ACID

Rationale:

1. **Service Autonomy:** Each team member can develop, test, and deploy independently without database-level coordination

2. **Technology Flexibility:** MongoDB for Prescription Service accommodates varying clinical document structures
3. **Fault Isolation:** Database failure in one service doesn't cascade to others
4. **Scalability:** Services scale databases independently (Appointment Service may need more read replicas than Patient Service)
5. **Bounded Contexts:** Enforces clean service boundaries (no accidental coupling via direct database access)

Consequences

Positive:

- **✓ Technology Choice:** Each service uses optimal database technology for its domain
- **✓ Fault Tolerance:** Prescription Service database crash doesn't prevent appointment booking
- **✓ Performance Optimization:** Each database tuned for specific query patterns (indexes, partitioning)
- **✓ Security Isolation:** Easier to apply different access controls per service
- **✓ Deployment Independence:** Database migrations scoped to single service

Negative:

- **✗ No Cross-Service Joins:** Cannot use SQL joins across patient, appointment, billing data. Must use API composition pattern with multiple asynchronous API calls to fetch related data.
- **✗ Distributed Transactions:** Cannot use ACID transactions across services. Mitigation: Implemented Saga pattern with compensating transactions and idempotent event handlers.
- **✗ Data Consistency Complexity:** Ensuring patient exists before creating appointment requires synchronous API call to Patient Service. If Patient Service is down, appointment creation fails (trade-off: availability vs. consistency).
- **✗ Data Duplication:** Billing Service caches patient names for invoices. If patient updates name, invoice displays old name until cache invalidated. Mitigation: Event-driven cache invalidation (listen to `patient.updated` events).
- **✗ Operational Overhead:** 4 databases to backup, monitor, upgrade, secure. Mitigated with Docker Compose and Kubernetes automation.
- **✗ Testing Complexity:** Integration tests require spinning up multiple databases (testcontainers for PostgreSQL and MongoDB instances with proper cleanup).

Implementation Strategy

Inter-Service Data Access:

1. **Synchronous Validation:** REST API calls when immediate consistency needed (e.g., verify patient exists)
2. **Asynchronous Updates:** RabbitMQ events for eventually consistent data (e.g., patient deleted → cascade delete appointments)
3. **Caching Layer:** Redis reduces cross-service calls for frequently accessed data

3.2 ADR-002: Hybrid Communication Patterns (Synchronous REST + Asynchronous Messaging)

Status: Accepted

Date: November 2025

Context

Microservices need to communicate for various purposes: data validation, event notifications, background processing, real-time queries. We needed to decide: **synchronous communication only (REST APIs) or asynchronous only (message queues) or both?**

Scenarios to Address:

1. User creates appointment → Must verify patient exists (immediate validation)
2. Appointment created → Generate invoice (can be background task)
3. Frontend queries patient details → Immediate response required
4. Patient deletes account → Cascade delete appointments and prescriptions
5. Payment received → Update appointment status

Alternatives Considered:

1. **Synchronous Only (REST APIs):** All inter-service calls via HTTP
2. **Asynchronous Only (Message Queues):** All communication via RabbitMQ
3. **Hybrid Approach:** Use both based on use case

Decision

We chose a **Hybrid Communication Architecture**:

Synchronous REST APIs for:

- User-facing operations requiring immediate response
- Data validation before state changes
- Real-time queries (get patient details, check appointment availability)

Asynchronous Messaging (RabbitMQ) for:

- Event notifications (appointment created, patient updated)
- Background processing (invoice generation, reports)
- Non-blocking workflows (cascade deletes, notifications)
- Decoupled service integration

Consequences

Positive:

- **✓ User Experience:** Synchronous APIs provide immediate feedback for user actions (appointment booking succeeds/fails instantly)
- **✓ System Resilience:** Asynchronous messaging allows services to be temporarily unavailable without losing events
- **✓ Scalability:** Message queues buffer high volumes during traffic spikes
- **✓ Decoupling:** New services can subscribe to existing events without modifying publishers
- **✓ Fault Tolerance:** RabbitMQ provides message persistence and automatic retries

Negative:

- **✗ Eventual Consistency:** Asynchronous operations mean data is eventually consistent. User Education: UI must indicate async operations ("Your invoice will be ready shortly").
- **✗ Debugging Difficulty:** Tracing request flow across sync and async boundaries
- **✗ Operational Overhead:** Must monitor and maintain RabbitMQ infrastructure (disk space for queued messages, connection limits, memory usage).

- **✗ Message Ordering:** Events may be processed out of order.
Mitigation: Design idempotent consumers (processing same event multiple times = same result).
- **✗ Testing Complexity:** Integration tests must handle asynchronous assertions with polling mechanisms to wait for eventual consistency (e.g., waiting up to 10 seconds for invoice generation after appointment creation).

3.2.1 Implementation Guidelines

When to Use Synchronous REST:

Use Case	Example	Rationale
Data validation	Verify patient exists before appointment	Prevent invalid state
User-facing queries	Get patient details	Immediate response required
Real-time state checks	Check appointment slot availability	Accurate current state needed
Transactional operations	Process payment	User needs confirmation

When to Use Asynchronous Messaging:

Use Case	Example	Rationale
Event notifications	<code>appointment.created</code>	Notify multiple subscribers
Background processing	Generate monthly reports	Long-running task
Cascade operations	<code>patient.deleted</code> → delete appointments	Non-blocking workflow
Integration with external systems	Send appointment reminder SMS	Decouple from external API latency

3.3 ADR-003: JWT Authentication with Centralized Gateway Validation

Status: Accepted

Date: November 2025

3.3.1 Context

Healthcare systems require secure authentication and authorization. With 4 microservices, we needed to decide: **How should users authenticate, and how should services verify user identity?**

Requirements:

- Secure user authentication (patients, doctors, administrators)
- Role-based access control (patients can only see their own records)
- Stateless authentication (supports horizontal scaling)
- Minimize inter-service authentication calls
- Single Sign-On (authenticate once, access all services)
- HIPAA security considerations (audit trail, session timeout)

Alternatives Considered:

1. **Session-Based Authentication:** Store sessions in Redis, validate on every request
2. **OAuth 2.0 with External Provider:** Use Google/Microsoft for authentication
3. **JWT with Per-Service Validation:** Each service validates JWT independently

4. **JWT with Centralized Gateway Validation:** Kong validates JWT, services trust gateway

3.3.2 Decision

We chose **JWT (JSON Web Tokens) with Centralized Gateway Validation:**

Architecture:

1. **Authentication Service:** Patient Service issues JWTs after successful login
2. **Centralized Validation:** Kong API Gateway validates JWT on every incoming request
3. **Service Trust Model:** Backend services trust tokens validated by gateway (no re-validation)
4. **Token Structure:** Contains user ID, role, expiration

Token Generation (Patient Service):

The login endpoint authenticates user credentials, then generates a JWT token containing: - Subject (user ID) - Email and role (for RBAC) - Expiration time (24 hours from issuance) - Issued-at timestamp and issuer claim. Token signed with shared JWT_SECRET using HS256 algorithm.

Service Trust (Billing Service):

Backend services receive X-User-Id and X-User-Role headers from Kong (after JWT validation). Services trust these headers and perform authorization checks (e.g., patients can only access their own invoices). No JWT parsing or validation needed in backend services.

3.3.3 Consequences

Positive:

-  **Stateless Authentication:** No session storage needed, enables horizontal scaling:
-  **Single Sign-On:** Users authenticate once, access all services:
-  **Performance:** No database lookup on every request (vs. session-based auth):
-  **Centralized Security:** Kong validates all tokens, backend services simplified: Services don't need JWT validation logic - they trust headers forwarded by Kong
-  **Audit Trail:** JWT claims provide context for logging
- Every log entry includes user context without database joins.
-  **Role-Based Access Control:** JWT contains role claim for authorization. Services check role (e.g., "doctor" can view all patient records, "patient" can only view own records) and raise Forbidden exception if unauthorized.

Negative:

-  **Token Revocation Complexity:** Cannot immediately invalidate tokens
-  **Token Size:** JWTs larger than session IDs (increases bandwidth)
-  **Secret Management:** Shared secret must be securely distributed
-  **Token Theft Risk:** Stolen token grants access until expiration
-  **Clock Synchronization:** Services need synchronized clocks for exp validation
-  **Cannot Update Claims:** Once issued, token claims immutable

3.3.4 Implementation Details

Security Best Practices Implemented:

1. **HTTPS Only:** Tokens transmitted over encrypted connections
2. **Short Expiration:** 15-minute access tokens, 7-day refresh tokens
3. **Token Blacklist:** Redis set for revoked tokens (checked by Kong)
4. **Refresh Token Rotation:** New refresh token issued on each refresh

5. Audit Logging: All authentication events logged with user context

4. Monitoring and Observability

A comprehensive monitoring stack provides visibility into system health, performance, and behavior in production. The system implements **Prometheus** for metrics collection and **Grafana** for visualization.

4.1 Grafana Dashboards

4.1.1 Billing Service Dashboard

Dashboard Components:

1. Overview Panel

- o Total requests/sec
- o Average response time
- o Error rate

2. Business Metrics Panel

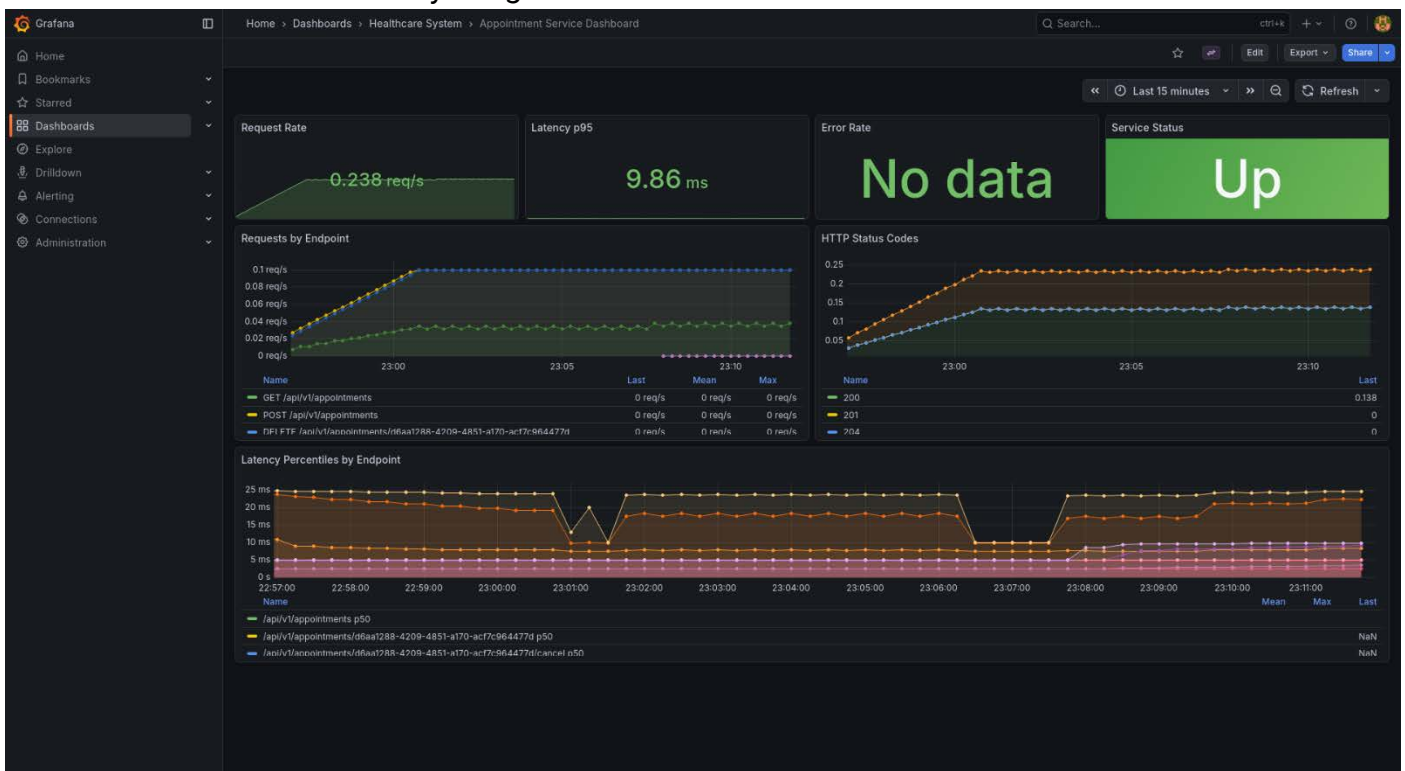
- o Invoices generated (rate)
- o Payments processed (rate)
- o Total revenue (counter)

3. Performance Panel

- o Request latency (percentiles: p50, p95, p99)
- o Requests by endpoint (bar chart)
- o Error rate by endpoint

4. Infrastructure Panel

- o Database connection pool usage
- o CPU and Memory usage



Sample PromQL Queries:

```
# Average response time
rate(billing_service_request_duration_seconds_sum[5m])
/ rate(billing_service_request_duration_seconds_count[5m])

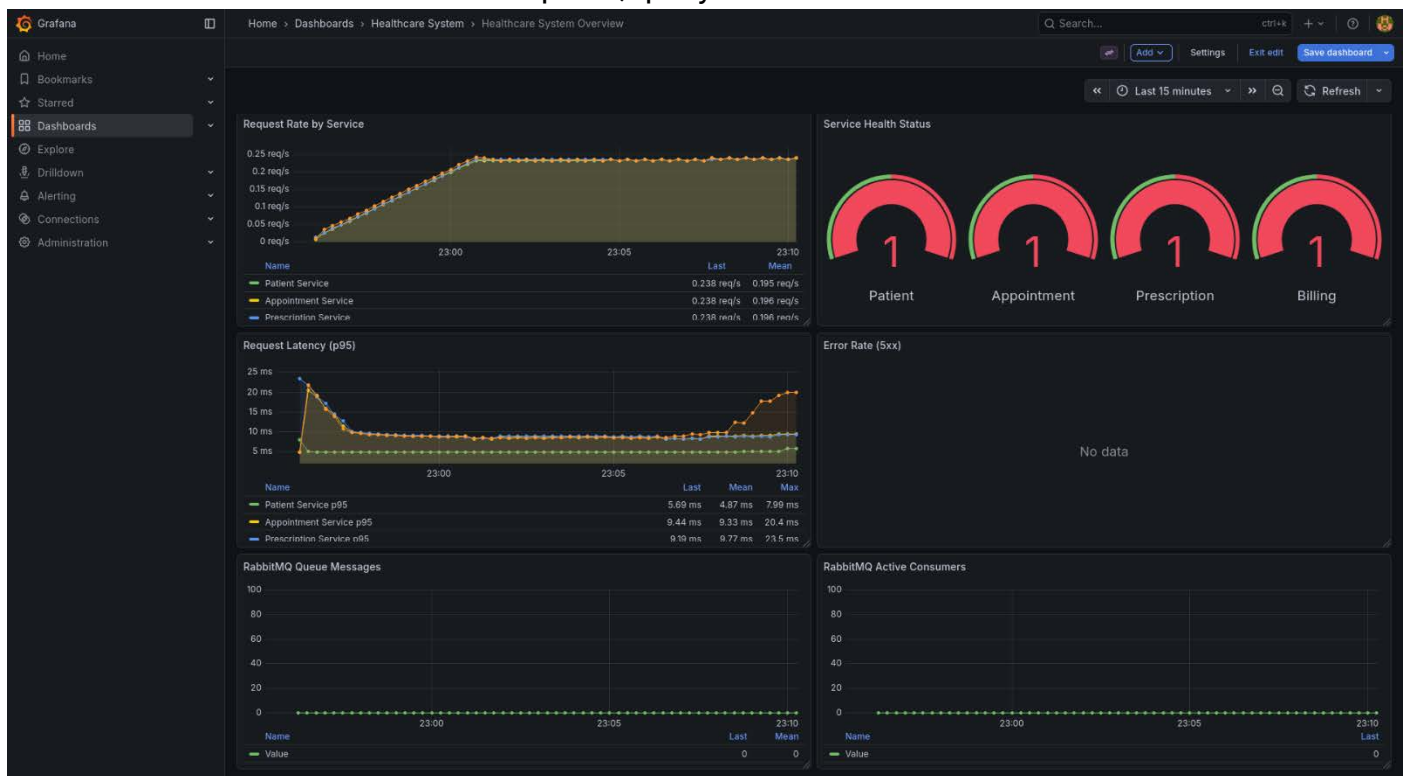
# Error rate percentage
sum(rate(billing_service_requests_total{status=~"5.."}[5m]))
/ sum(rate(billing_service_requests_total[5m])) * 100

# Invoices generated per minute
rate(billing_invoices_generated_total[1m]) * 60
```

4.1.2 System Overview Dashboard

Cross-Service Metrics:

- **Service Health:** Up/down status for all services
- **Request Volume:** Total requests across all services
- **Error Rates:** Comparison across services
- **Latency Comparison:** Response times by service
- **RabbitMQ Metrics:** Message rates, queue depths, connections
- **Database Metrics:** Connection pools, query times



4.2 Observability Best Practices Implemented

- **RED Metrics:** Rate, Errors, Duration tracked for all services
- **Golden Signals:** Latency, Traffic, Errors, Saturation
- **Distributed Tracing:** Correlation IDs in logs (OpenTelemetry future)
- **Health Checks:** Kubernetes liveness/readiness probes
- **Alerting:** Prometheus alerts for critical issues

Prometheus

Query

Alerts

Status > Target health

Select scrape pool

Filter by target health

Filter by endpoint or labels

appointment-service

1 / 1 up

Endpoint

Labels

Last scrape

State

<http://appointment-service:8002/metrics>

instance="appointment-service:8002"

job="appointment-service"

service="appointment-service"

tier="microservice"

9.761s ago

6ms

UP

billing-service

1 / 1 up

Endpoint

Labels

Last scrape

State

<http://billing-service:8004/metrics>

instance="billing-service:8004"

job="billing-service"

service="billing-service"

tier="microservice"

765ms ago

7ms

UP

patient-service

1 / 1 up

Endpoint

Labels

Last scrape

State

<http://patient-service:8001/metrics>

instance="patient-service:8001"

job="patient-service"

service="patient-service"

tier="microservice"

8.633s ago

5ms

UP

prescription-service

1 / 1 up

Endpoint

Labels

Last scrape

State

<http://prescription-service:8003/metrics>

instance="prescription-service:8003"

job="prescription-service"

service="prescription-service"

tier="microservice"

7.583s ago

4ms

UP

prometheus

1 / 1 up

Endpoint

Labels

Last scrape

State

<http://localhost:9090/metrics>

instance="localhost:9090"

job="prometheus"

service="prometheus"

9.15s ago

8ms

UP

rabbitmq

1 / 1 up

Endpoint

Labels

Last scrape

State

<http://rabbitmq:15692/metrics>

instance="rabbitmq:15692"

job="rabbitmq"

service="rabbitmq"

tier="infrastructure"

25.06s ago

33ms

UP

5. Student Contributions

This section documents individual contributions to the project, highlighting the modular development approach where each team member owned a complete microservice.

5.1 Sai Deep Kandra (25207532): Patient Management Service

Key Deliverables:

- Patient registration and authentication (JWT token generation and validation)
- Medical history management
- FHIR Patient resource implementation, search and retrieval functionality
- PostgreSQL database schema design, migrations and initialization scripts
- Frontend web application
- Integration with Kong API Gateway for authentication

5.2 Lakshmi Kiran Chikka Hosagavi Manchaiah (25207663): Appointment Scheduling Service

Key Deliverables:

- Doctor availability management
- Slot availability checking algorithm and conflict detection
- FHIR Appointment resource implementation, booking and cancellation
- RabbitMQ event publishing for appointment lifecycle
- Frontend web application
- Redis caching for availability queries

5.3 Akshay Channapla Udaya Kumar (25211966): Prescription Management Service

Key Deliverables:

- Prescription creation and management
- Medication history tracking
- Prescription refill request workflow
- FHIR MedicationRequest resource implementation
- MongoDB document design for clinical records
- Integration with Patient Service for verification

5.4 Vijeth Prashanth Hegde (25202022): Billing Service and Monitoring Infrastructure

Key Deliverables:

- Automated invoice generation and payment processing with validation and status tracking
- Billing Service implementation with FHIR Invoice resource
- Prometheus monitoring setup for all services
- Grafana dashboard development
- Custom metrics instrumentation (business + technical metrics)
- Alerting rules for critical system events

5.5 Collaborative Work

Shared Components (Team Effort):

- common/ directory (auth, logging, messaging, exceptions, middleware, utils)
- docker-compose.yml (full-stack orchestration)
- kubernetes/ manifests (deployment configurations)
- kong/kong.yml (API Gateway configuration)
- .github/ instructions and documentation
- Architecture diagrams and ADRs

Integration Testing (Team Effort):

- scripts/e2e_test_all_services.sh (comprehensive E2E testing)
- Cross-service API contracts
- RabbitMQ event schemas and routing keys

Documentation (Team Effort):

- System architecture documentation
- API endpoint documentation (OpenAPI)
- Deployment guides (local, Docker, Kubernetes)
- Troubleshooting guides

6. Trade-off Analysis and Lessons Learned

This section reflects on the architectural decisions made, their real-world impact during implementation, and what we would do differently for a production system.

6.1 Major Trade-offs Revisited

6.1.1 Database Per Service

✓ What Worked Well:

- Team members developed independently without database conflicts or coordination overhead
- Technology flexibility allowed MongoDB for clinical documents (prescriptions) while using PostgreSQL for transactional data
- Database failures isolated to single service (appointment DB issues didn't affect billing)

✗ What Was Challenging:

- Cross-service queries required multiple API calls (performance overhead)
- Debugging distributed transactions was complex (Saga pattern learning curve)
- Integration tests needed 4 separate database instances (slow test startup)

Lessons Learned:

- **Event-driven cache invalidation is hard:** We initially missed invalidating cached patient names when patients updated their profiles, showing stale names on invoices. Fixed by listening to `patient.updated` events.
- **Idempotency is critical:** RabbitMQ occasionally redelivered `appointment.created` events, creating duplicate invoices until we added `appointment_id` unique constraint.
- **Transaction boundaries matter:** Initially tried to update appointment and create invoice in one transaction (failed due to separate DBs). Learned to embrace eventual consistency.

6.1.2 Synchronous + Asynchronous Communication

✓ What Worked Well:

- User-facing operations (appointment booking) responded immediately via REST
- Background processing (invoice generation) didn't block user requests
- RabbitMQ persisted events when billing service was down (resilience)
- Adding new event consumers (future notification service) didn't require publisher changes

✗ What Was Challenging:

- Debugging event flows required correlation IDs across logs
- Testing async workflows needed polling/waiting patterns (brittle tests)

Lessons Learned:

- **Dead Letter Queues save debugging time:** Initially lost failed events silently. Adding DLQ allowed manual inspection.
- **Message ordering is not guaranteed:** Tried to rely on event order (appointment created → cancelled), but events arrived out of order. Made consumers idempotent instead.

6.1.3 JWT with Centralized Gateway Validation

✓ What Worked Well:

- Services simplified (no JWT validation code duplicated)
- Horizontally scaled services without shared session store
- Centralized security policy enforcement at gateway
- Audit logging captured all authenticated requests in one place (Kong logs)

✗ What Was Challenging:

- Token revocation required blacklist in Redis (undermined stateless benefits)
- Rotating JWT secret required coordinating Kong + Patient Service deployment
- Debugging authentication failures required checking both Kong and service logs
- Local development mode (no Kong) required services to validate tokens themselves

Lessons Learned:

- **Short expiration is the real solution:** Instead of complex token revocation, used 15-minute access tokens with refresh tokens (forces re-authentication frequently)
- **Correlation IDs track requests:** Added correlation ID from Kong to track requests through service chain

6.2 Performance Considerations

Bottlenecks Identified:

- **Cross-service REST calls:** Appointment Service → Patient Service validation adds 50-100ms latency
 - Mitigation: Redis caching reduced to 5-10ms for cached patients
- **RabbitMQ message processing lag:** Invoice generation took 3-5 seconds during peak load
 - Mitigation: Scaled Billing Service to 3 replicas, processed messages in parallel

Optimizations Implemented:

- Database indexes on frequently queried columns (patient email, invoice patient_id)
- Connection pooling for PostgreSQL (max 20 connections per service)
- Redis caching for read-heavy operations (patient details, appointment availability)
- Async I/O throughout (FastAPI + httpx + aio-pika)

Not Implemented (Future Work):

- Database read replicas for query scaling
- CDN for static assets
- API response compression
- Database query optimization (EXPLAIN ANALYZE)

6.3 Key Takeaways

What We Learned:

1. **Distributed systems are complex:** What seems simple (create appointment → generate invoice) becomes complex with network failures, eventual consistency, and error handling
2. **Testing strategy matters:** Unit tests are fast but integration tests caught real issues (database constraints, API contract mismatches)
3. **Observability is not optional:** Without Prometheus/Grafana, we would have been blind to performance issues
4. **Shared components reduce duplication:** common/ directory reduced code across services

What We Would Do Differently:

1. **Start with modular monolith:** Build all 4 services in one codebase initially, split into microservices only when scaling needs emerged

2. **Implement distributed tracing earlier:** Correlation IDs helped, but OpenTelemetry would have visualized request flows better
3. **More comprehensive E2E tests:** Caught bugs late that E2E tests would have found earlier
4. **Use managed services for infrastructure:** Self-hosting RabbitMQ/Redis was educational but time-consuming; managed services (AWS RDS, CloudAMQP) would have been faster

What Validated Our Approach:

- Database-per-service enabled true parallel development
 - FHIR compatibility made schemas extensible
 - Docker Compose provided reproducible environments
 - Prometheus/Grafana caught performance regressions early
-

7. Conclusion

This project successfully demonstrates the design and implementation of a distributed healthcare management system, applying core principles of microservices architecture, event-driven communication, and cloud-native deployment. The system balances academic learning objectives with practical architectural trade-offs.

7.1 Objectives Achieved

- **Distributed System Implementation:** 4 independent microservices with clear service boundaries (Patient, Appointment, Prescription, Billing)
- **Architectural Trade-off Analysis:** Documented 3 major decisions (Database Per Service, Hybrid Communication, JWT Authentication) with explicit consequences
- **Cloud-Native Deployment:** Successfully deployed on Docker Compose (integration testing) and Kubernetes (production-like orchestration)
- **Fault Tolerance:** Implemented retries with exponential backoff, health checks, graceful shutdown, and message persistence
- **Security:** JWT-based authentication, API Gateway centralization, password hashing, RBAC authorization
- **Observability:** Comprehensive monitoring with Prometheus metrics, Grafana dashboards, structured logging, and alerting
- **Reproducibility:** Clear documentation, automated scripts, and step-by-step deployment guides ensure anyone can run the system

7.2 Key Technical Contributions

Architectural Patterns Demonstrated:

- Database-per-Service pattern for service autonomy
- API Gateway pattern for centralized concerns
- Event-Driven Architecture with RabbitMQ
- Saga pattern for distributed transactions
- Repository pattern for data access
- Dependency Injection for testability

7.3 Learning Outcomes

Distributed Systems Concepts:

- Eventual consistency vs. strong consistency trade-offs
- CAP theorem implications (chose Availability over Consistency for non-critical paths)
- Network partition handling (retries, timeouts, circuit breakers)
- Distributed transaction complexity (Saga pattern)
- Service discovery and load balancing

Operational Skills:

- Container orchestration with Docker Compose and Kubernetes
- Monitoring and alerting setup
- Debugging distributed systems (correlation IDs, distributed tracing concepts)
- Deployment automation (bash scripts for CI/CD-like workflows)

7.4 Project Scope and Limitations

Within Scope (Delivered):

- 4 functional microservices with REST APIs
- FHIR-compatible data models
- Database-per-service implementation
- Synchronous and asynchronous communication
- JWT authentication and authorization
- Three deployment methods (local, Docker, Kubernetes)
- Unit, integration, and E2E tests
- Detailed documentation and ADRs
- Frontend web application (Next.js 14 with TypeScript)

Out of Scope (Future Extensions):

- Full FHIR R4 compliance (used compatible schemas)
- External integrations (payment gateways, email/SMS)
- Production-grade secrets management (Vault, AWS Secrets Manager)
- Database replication and high availability
- Advanced service mesh (Istio, Linkerd)
- Comprehensive disaster recovery strategy

7.5 Final Reflection

This project successfully demonstrates that **distributed systems require careful trade-off analysis at every decision point**. There are no universal “best practices”: only the **least worst combination of trade-offs** for a given context.

Key Insight: The value of this project lies not in achieving perfection, but in **understanding the consequences of our architectural decisions**. Every pattern we chose (database-per-service, hybrid communication, JWT auth) came with costs and benefits. The ADRs document these trade-offs explicitly, ensuring future developers understand the rationale.

Acknowledgments: This project benefited from open-source tools, online documentation, and distributed systems literature. Special thanks to the FastAPI, SQLAlchemy, RabbitMQ, Prometheus, and Kubernetes communities for excellent documentation and tooling.